

Fast First-Hit Determination for Large and Composite Sparse Volumetric Data Structures with Mesh Shaders

Mehmet Oguz Derin
Morgenrot
Izmir, Turkiye
oguz@morgenrot.net

Takahiro Harada
Advanced Micro Devices, Inc. (AMD)
Santa Clara, USA
Morgenrot
Santa Clara, USA
takahiro.harada@amd.com

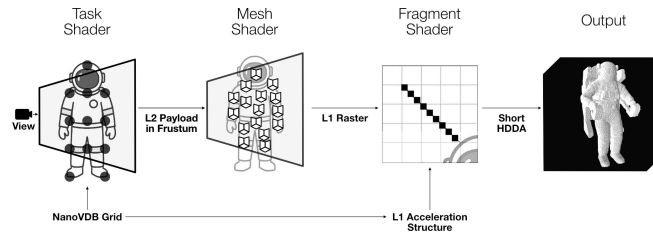


Figure 1: Our first-hit rendering pipeline for sparse grids. A task shader expands view-culled Level 2 (L2) nodes into Level 1 (L1) candidates. Then, a mesh shader generates rasterizable proxies for these nodes. Finally, a fragment shader executes a short Hierarchical Digital Differential Analyzer (HDDA) traversal within each rasterized node to find the first hit, using an acceleration structure over L1.

Abstract

Sparse NanoVDB volumes are increasingly used for interactive effects and scientific visualization. However, fast primary-visibility (first-hit) computation typically requires either compute-centric pipelines with multi-draw indirect and transient geometry buffers, or pure ray marching/HDDA with significant empty-space work. We introduce a rasterization-centric pipeline that maps the NanoVDB hierarchy directly onto modern task and mesh shaders, eliminating intermediary buffers while tightly coupling visibility culling with shading. A task shader expands only potentially visible Level 2 Upper Internal nodes into candidate Level 1 Lower Internal nodes for the current view; a mesh shader materializes just the faces likely to contribute to the frontmost G-buffer hit, passing each tile’s world-space bounds to the fragment stage to shorten per-pixel traversal. To further reduce empty-space stepping, we build an acceleration structure over Level 1 and use it to jump HDDA between nodes; the same structure is queried with Vulkan ray queries for shadows and selective skipping. Our implementation requires no modifications to NanoVDB data (only runtime auxiliaries such as the tile AS and per-frame bounds) and runs on commodity GPUs. Across a diverse set of scenes, including open Vox and VDB datasets, our approach achieves $1.5\times$ end-to-end speedups on average over vertex-shader baselines while preserving first-hit G-buffer fidelity. We argue that task-mesh pipelines provide a practical, intermediary-buffer-free

pathway to interactive first-hit NanoVDB rendering suitable for integration into existing Vulkan engines.

CCS Concepts

• **Computing methodologies** → **Rasterization; Volumetric models; Visibility.**

ACM Reference Format:

Mehmet Oguz Derin and Takahiro Harada. 2025. Fast First-Hit Determination for Large and Composite Sparse Volumetric Data Structures with Mesh Shaders. In *SIGGRAPH Asia 2025 Posters (SA Posters '25)*, December 15–18, 2025, Hong Kong, Hong Kong. ACM, New York, NY, USA, 2 pages. <https://doi.org/10.1145/3757374.3771422>

1 Introduction

VDB data structures are increasingly pervasive across VFX and visualization pipelines because they deliver high spatial locality and scalable storage [Zellmann, Jaros, Amstutz, and Wald 2025; Sarton, Zellmann, Demirci, Gudukbay, Alexandre-Barff, Lucas, Discher, Wesner, and Wald 2023]. First-hit determination—the frontmost fragment in the camera direction—is a key operation for deferred lighting, depth peeling, and hybrid rendering. State-of-the-art solutions often favor compute/ray tracing (e.g., HDDA-based and RTX BVHs) [Williams, Huang, Swartz, Klár, Thakkar, Cong, Ren, Li, Fuji-Tsang, Fidler, Sifakis, and Museth 2024; Wald, Zellmann, and Morrical 2021], or they rely on rasterization with a traditional vertex-shader pipeline with large proxy geometries. Both paths incur overheads: transient buffers and poor culling coherence for the former; empty-space work and rasterization for the latter.

NanoVDB provides a GPU-friendly, pointerless layout of VDB trees with good cache behavior and per-node statistics [Museth 2021]. Recent work has shown that fixed-depth VDB with HDDA traversals delivers fast sparse sampling and large-scene scalability

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for third-party components of this work must be honored. For all other uses, contact the owner/author(s).

SA Posters '25, Hong Kong, Hong Kong

© 2025 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-2134-2/25/12

<https://doi.org/10.1145/3757374.3771422>

Table 1: Rendering performance at 1080p on 4060 Laptop

Scene (VDB)	Baseline (FPS)	Our Method (FPS)
Crawler	42	67
Space	307	459

in deep learning settings [Williams, Huang, Swartz, Klár, Thakkar, Cong, Ren, Li, Fuji-Tsang, Fidler, Sifakis, and Museth 2024]. Triangulated active-region boundaries enable accelerated empty-space skipping [Wald, Zellmann, and Morrical 2021]. Mesh shaders have recently been used to convert volumetric content into efficient rasterizable proxies [Tricard 2024]. VDB-based volume rendering on GPUs, including compressed variants, demonstrates the practicality of VDB in interactive and production contexts [Museth 2021; Zellmann, Jaros, Amstutz, and Wald 2025]. For a broad perspective on sparse volume visualization strategies, see [Sarton, Zellmann, Demirci, Gudukbay, Alexandre-Barff, Lucas, Discher, Wesner, and Wald 2023]. These approaches target either the Vulkan ray tracing model for shadowing/queries or pure compute pipelines, but typically not a tight integration with rasterization.

We propose a mesh-shader pipeline that directly maps NanoVDB’s hierarchy to task–mesh stages. Our approach (1) avoids intermediary geometry buffers, (2) emits only faces that are likely to contribute to the first hit, and (3) couples per-pixel traversal with a tile acceleration structure (AS) to leap among occupied regions. The result is a Vulkan implementation that attains end-to-end speedups over the vertex-shader raster pipeline on commodity GPUs, while preserving G-buffer equivalence. Our primary contributions are: (1) a task–mesh mapping of NanoVDB that expands only potentially visible L2 nodes and materializes minimal L1 nodes for first-hit rendering, and (2) an AS over L1 nodes that enables node-to-node HDDA jumps in the fragment stage and integrates naturally with Vulkan ray queries for shadows/skip tests.

2 Method

Given a NanoVDB grid [Museth 2021], in each frame:

- **Expansion.** The task shader frustum-culls L2 nodes and estimates screen-space bounds for their L1 nodes. Only tiles with projected area above a tunable threshold are enqueued. Occlusion and view-cone tests prune back-facing tiles.
- **Triangularization.** For each candidate L1 tile, mesh shader emits only faces whose outward normals may contribute to frontmost hit for the current view. Each emitted face carries tile index and its world-space AABB; per-face payloads (IDs, bounds) eliminate transient vertex/index buffers.

The fragment shader receives the tile AABB and tile ID. It computes the camera ray segment clipped to the tile AABB and performs a short-range HDDA inside the tile to detect the first non-empty sample contributing to the G-buffer.

We build an AS over L1 AABBs to skip empty space when initializing or continuing HDDA with hardware-accelerated ray queries. If the clipped segment in the current tile yields no hit, we advance to the next intersected tile via AS traversal and repeat the short-range HDDA. We treat NanoVDB as read-only. The only runtime auxiliaries are (i) per-frame L1/L2 bounds and view-space filters,

and (ii) the tile AS with small AABB and index buffers. In contrast to compute-centric pipelines, no multi-draw indirect lists or transient geometry buffers are required.

3 Implementation

We implement the pipeline in Vulkan 1.3 using task/mesh shaders and fragment shaders for G-buffer writes. The NanoVDB tree is resident in device memory and accessed through a storage buffer binding. The tile AS is built during load-time BVH over AABBs using driver-provided Vulkan extension. Our codepaths are designed to coexist with existing engine resource lifecycles and render graphs.

4 Results

We evaluate on RTX 3060/4060 across a suite of publicly available VDB volumes and composite scenes mixing multiple grids, with constant resolution of 32844x24702x9156 under changing sparsity, with 1.69MB Vulkan AS memory overhead on average, while keeping screen occupancy of volume above 10% for all frames. We compare against rasterization of L1 node proxies using a vertex shader pipeline while keeping the fragment shader constant across both pipelines to measure the impact of the geometry stages. Our method attains 1.5× end-to-end speedups on average relative to the vertex shader baseline, while preserving first-hit G-buffer depth within numerical tolerance. Our ablations show that task-shader L2 enumeration reduces mesh-shader divergence, AS-enabled jumping reduces empty-space work, and emitting only visible faces has a negligible impact on fidelity with dot product instructions.

5 Conclusion

We presented a task–mesh shader pipeline for first-hit determination in NanoVDB scenes, complements volume methods [Wald, Zellmann, and Morrical 2021]. By expanding primitives directly on the GPU, materializing only potentially visible tile faces, shortening per-pixel traversal, and jumping across tiles with an AS (also reused for Vulkan ray queries), we achieve, on average, 1.5× end-to-end speedups over raster/fragment baselines on commodity GPUs, without modifying NanoVDB data, enabling more stable experience for refresh-rate sensitive display environments such as VR. We believe this provides a practical pathway for integrating sparse volumes into Vulkan engines that already deploy mesh shaders.

References

- K. Museth. NanoVDB: A GPU-Friendly and Portable VDB Data Structure For Real-Time Rendering And Simulation. In *SIGGRAPH Talks*, 2021.
- F. Williams, J. Huang, J. Swartz, G. Klár, V. Thakkar, M. Cong, X. Ren, R. Li, C. Fuji-Tsang, S. Fidler, E. Sifakis, and K. Museth. fVDB: A Deep-Learning Framework for Sparse, Large-Scale, and High-Performance Spatial Intelligence. *ACM Transactions on Graphics*, 43(4), 2024.
- I. Wald, S. Zellmann, and N. Morrical. Faster RTX-accelerated empty space skipping using triangulated active region boundary geometry. In *Eurographics Symposium on Parallel Graphics and Visualization (PGV)*, 2021.
- T. Tricard. Interval Shading: using Mesh Shaders to generate shading intervals for volume rendering. *Proceedings of the ACM on Computer Graphics and Interactive Techniques*, 7(3), 2024.
- S. Zellmann, M. Jaros, J. Amstutz, and I. Wald. GPU Volume Rendering with Hierarchical Compression Using VDB. In *SIGGRAPH*, 2025.
- J. Sarton, S. Zellmann, S. Demirci, U. Gudukbay, W. Alexandre-Barff, L. Lucas, J. M. Discher, S. Wesner, and T. Wald. State-of-the-art in Large-Scale Volume Visualization Beyond Structured Data. *Computer Graphics Forum*, 42(3), 2023.